Control y Automatización de una Grúa Portacontenedores Portuaria (STS)

Proyecto Global Integrador

Peña Lautaro (13099) Peralta Bruno (13220)

2026

Autómatas y Control Discreto
Ing. Mecatrónica — UNCuyo

Contenido

1. Introducción

2. La planta (provista)

3. Nivel 2 — Control regulatorio

4. Nivel 1 — Control supervisor (DEDS)

5. Nivel 0 — Protección

6. Implementación en CODESYS

7. Simulación y resultados

8. Conclusiones

Introducción

El sistema: grúa portacontenedores STS

- Grúa de muelle tipo pórtico (*Ship-to-Shore*) para carga/descarga de contenedores en barcos.
- Dos movimientos principales, simultáneos, en un plano vertical (2D):
  - **Carro** (*Trolley*): traslación horizontal.
  - **Izaje** (*Hoist*): subida/bajada de la carga.
- Carga suspendida que se **balancea como un péndulo** $\Rightarrow$ ejes acoplados.
- Objetivo: **control de accionamientos** y **operación semi-automática coordinada**.



Fuente: IMPSA

Arquitectura de control jerárquica

Sistema híbrido de control y protección — 3 niveles

Nivel	Rol	Tipo / T_s
2	Regulatorio (torque/velocidad)	Continuo muestreado — 1 ms
1	Supervisor (modos, coordinación)	**Eventos discretos** — 20 ms
0	Protección (seguridad)	Eventos discretos — 20 ms

- Jerarquía creciente: el Nivel 0 **sobrescribe** a los superiores ante falla.
- La **planta** (modelo físico y parámetros) es provista por la cátedra; el aporte propio está en el **diseño de los 3 niveles** y su **implementación**.

La planta (provista)

Modelo dinámico híbrido de la planta

La cátedra entrega el modelo completo. Es **híbrido**: combina dinámicas continuas con cambios discretos de estado.

Variables continuas (flow)

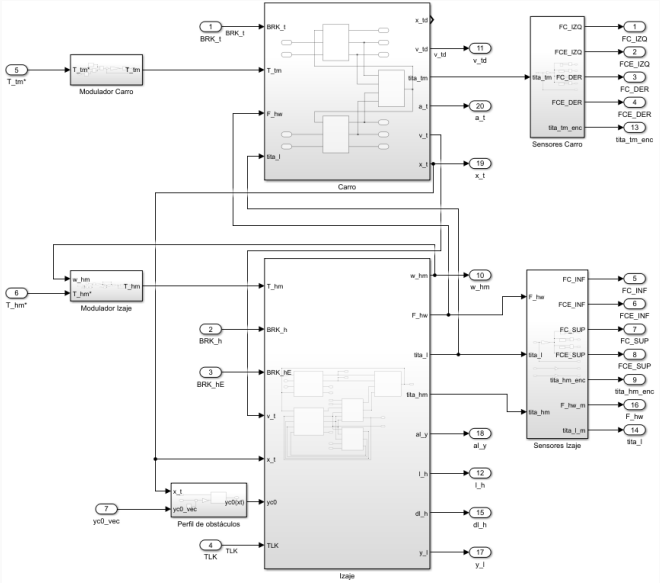
- Posición/velocidad de carro x_t, v_t e izaje y_h, v_h.
- Ángulo de balanceo θ_l, tensión de cable F_{hw}.
- Torques, fuerzas de contacto.

Estados discretos (jump)

- Twistlocks TLK: abierto/cerrado.
- Contacto: suspendido/apoyado.
- Cable: tenso/flojo.
- Frenos NC *BRK*: abierto/cerrado.

Carro e izaje acoplados por la carga pendular; interacción con el perfil de obstáculos del entorno.

Implementación de la planta en Simulink



Parámetros (dados)

M_t	30 000 kg
M_s	15 000 kg
$M_{c\,\text{máx}}$	50 000 kg
i_t / i_h	30/22
$v_{t\,\text{máx}}$	4 m/s
v_h	1,5–3 m/s

Sensores: encoders, fines de carrera, celda de carga, ángulo de spreader, LIDAR.

Nivel 2 — Control regulatorio

Nivel 2: objetivo

- Genera los comandos de **torque** T^* a partir de referencias de **velocidad** del supervisor.
- Control en cascada: **velocidad** (PI) $\rightarrow$ modulador de torque.
- Atenúa perturbaciones: balanceo de la carga y variaciones de masa.
- Opera muestreado en tiempo discreto.

Componentes diseñados

Lazo PI de velocidad • Observador de estado • Control anti-balanceo • Anti-windup • Compensación de gravedad

Lazo de velocidad: PI por asignación de polos

Planta simplificada del carro:

$$G_{vel,t}(s) = \frac{V_t(s)}{T_{tm}(s)} = \frac{K_f}{M_t\, s + b_t} \approx \frac{K_f}{M_t\, s}$$

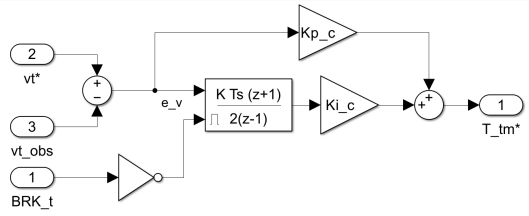
Controlador:

$$C_{vel}(s) = K_p + \frac{K_i}{s}$$

Comparando con $s^2 + 2\zeta\omega_n s + \omega_n^2$:

$$K_p = \frac{2\zeta\omega_n M_t}{K_f}, \quad K_i = \frac{\omega_n^2 M_t}{K_f}$$

$\omega_n = 0{,}8$ rad/s, $\zeta = 1$ (sin sobrepaso).



Anti-windup: la señal de freno resetea el integrador mientras el freno está cerrado.

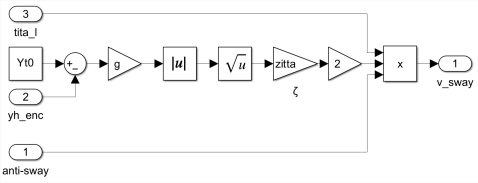
Control anti-balanceo (anti-sway)

Estrategia **desacoplada**: la velocidad del carro inyecta amortiguamiento sobre el péndulo.

$$v_{sway}(t) = K_{sway}\,\theta_l(t)$$

Sintonía por linealización local ($\omega_n = \sqrt{g/l_h}$):

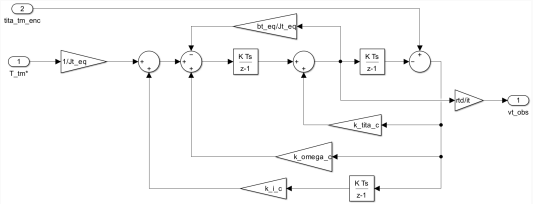
$$\boxed{K_{sway}(l_h) = 2\sqrt{g\,l_h}}$$



- **Gain scheduling** con la altura de izaje l_h.
- Válido sin aproximación de ángulos pequeños ($\cos\theta_l > 0$).
- Se subamortigua ($\zeta = 0{,}5$) para no sobrecorregir la posición.

Observador de estado con acción integral

- Solo se mide **posición** (encoder); el PI necesita la **velocidad**.
- Observador que estima $\tilde{\omega}$ a partir de θ.
- **Acción integral** sobre el error de observación $\Rightarrow$ elimina el error en régimen ante perturbaciones constantes (balanceo).
- Sintonía por separación de anchos de banda: $20\times$ más rápido que la planta.



Izaje: gravedad y límite de potencia

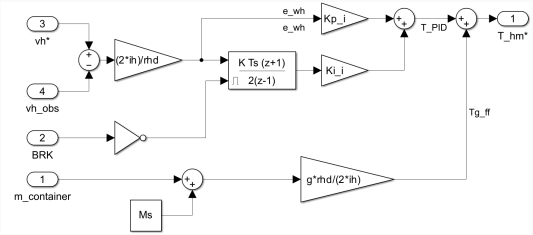
Compensación de gravedad (feedforward)

$$T_{g,ff} = \frac{m_l \, g \, r_{hd}}{2 \, i_h}$$

El PI sólo corrige errores de modelo; el peso estático lo sostiene el feedforward.

Operación a potencia constante

El límite de velocidad de izaje se reduce con la masa estimada para no exceder la potencia nominal.



Nivel 1 — Control supervisor (DEDS)

Nivel 1: el cerebro lógico de la grúa

Autómata de **eventos discretos** (Stateflow), $T_{s1} = 20$ ms. Organizado en **procesos paralelos** que se evalúan simultáneamente:

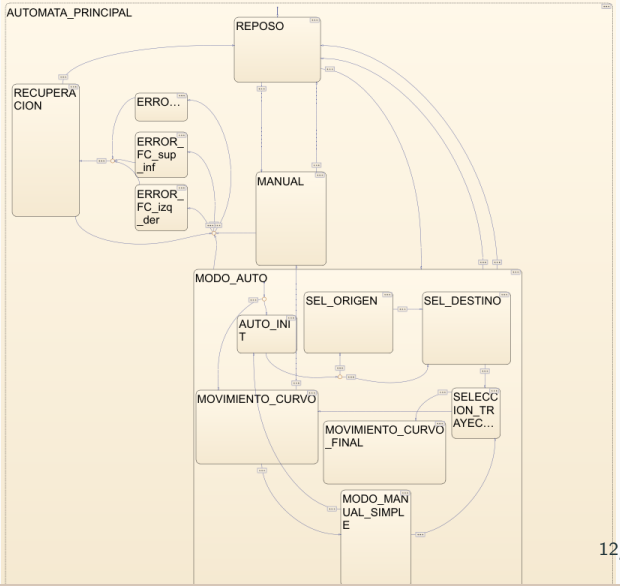
- **Autómata principal** de modos.
- Gestión de **twistlocks**.
- Análisis espacial: **perfil e índices** (LIDAR).
- **Suavizado** de consignas (jerk/acel.).
- **Límites suaves** de velocidad.
- **Watchdog** de comunicación.

Entradas / Salidas

Sensores + joystick + fallas $\rightarrow$ consignas vx_cmd/vy_cmd, frenos *BRK*, TLK, vh_lim, heartbeat.

Autómata principal de modos de operación

- **Reposo**: detenido, frenos cerrados.
- **Manual**: control directo por joystick.
- **Automático**: ciclo autónomo de carga/descarga.
- **Recuperación**: resguardo tras falla.
- **Errores**: por tipo de fin de carrera.



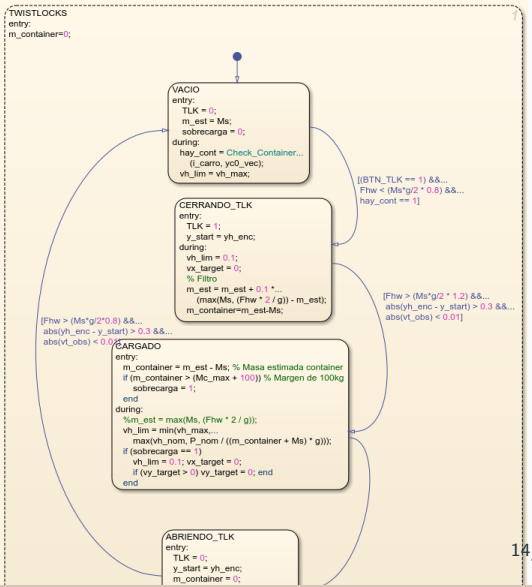
Modo automático: ciclo coordinado sin colisiones

- Secuencia **origen** $\rightarrow$ **destino** gestionada por el estado del ciclo y los índices seleccionados.
- Una **función planificadora** genera las consignas de velocidad (vx_target, vy_target) coordinando ambos ejes.
- Usa la **envolvente de seguridad** del perfil para **evadir colisiones** con la pila de contenedores.
- Movimientos **simultáneos** (izaje + traslación) para optimizar el tiempo de ciclo.
- **Transferencia suave** a manual si el operador interviene (*bumpless*).

Gestión de twistlocks y estimación de carga

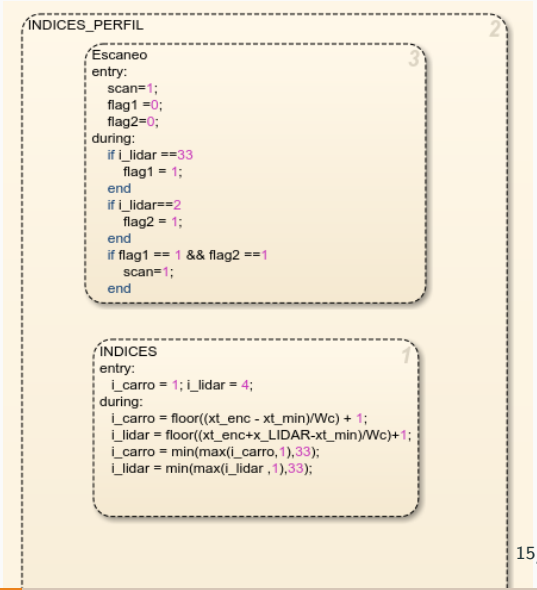
Cuatro estados: `VACIO` → `CERRANDO` → `CARGADO` → `ABRIENDO`.

- Toma/libera la carga sólo con el spreader **apoyado** (interlock).
- **Estima la masa** del contenedor con la celda de carga.
- Detecta **sobrecarga** y limita la velocidad por curva de potencia (vh_lim).



Análisis espacial: perfil de obstáculos e índices

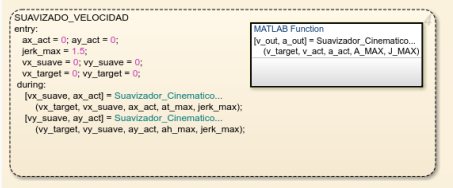
- El **LIDAR** releva el perfil de alturas bajo el área de trabajo.
- Calcula los **índices discretos** del carro y la cruceta.
- Genera una **envolvente de seguridad** (márgenes laterales/superiores).
- Habilitación del modo automático: perfil **escaneado por completo**.



Suavizado y límites suaves de velocidad

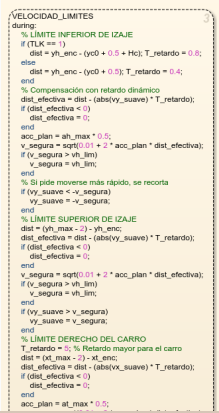
Suavizado (jerk/aceleración)

Convierte consignas crudas en vx_suave/vy_suave graduales; clave en modo manual.



Límites suaves (frenado anticipado)

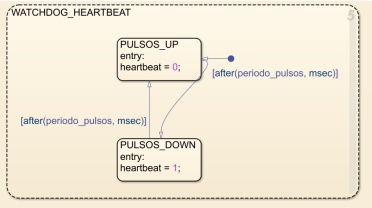
$v_{segura} = \sqrt{2\,a\,d}$ con retardo dinámico: recorta la velocidad cerca de los límites, antes de los fines de carrera.



Watchdog y manejo del joystick

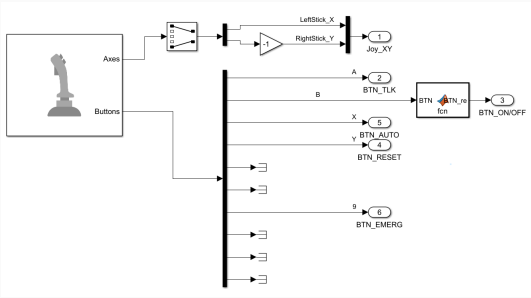
Watchdog (heartbeat)

- Emite un latido periódico al Nivel 0.
- Su ausencia ⇒ pérdida de comunicación ⇒ parada de emergencia.



Joystick y botones

- Ejes → velocidad de carro e izaje.
- Botones → TLK, modo manual/auto, reset, emergencia.



Nivel 0 — Protección

Nivel 0: capa de máxima autoridad

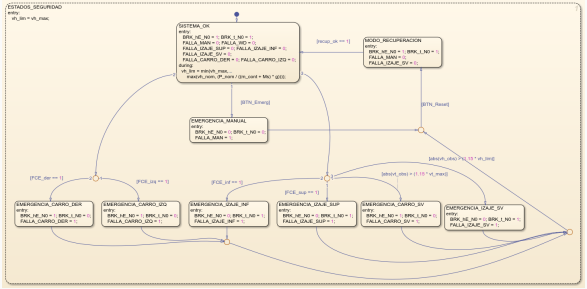
- Supervisa la planta de forma **permanente** (redundancia de sensores).
- Ante anomalía, **sobrescribe** a los niveles superiores y aplica los frenos.
- Recomputa vh_lim de forma **independiente** (defensa en profundidad).

Condiciones de emergencia

Sobrevelocidad • fines de carrera de emergencia • parada manual (botón) • pérdida de *heartbeat*.

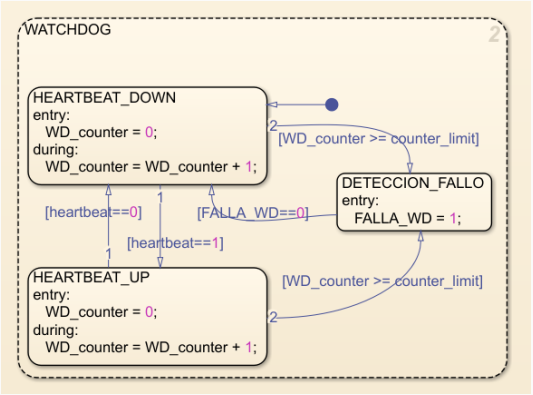
Autómata de seguridad: frenado selectivo

- Estado nominal `SISTEMA_OK` (ambos frenos abiertos).
- **Frenado selectivo**: sólo se detiene el **eje afectado**.
  - Falla izaje ⇒ frena izaje, carro libre.
  - Falla carro ⇒ frena carro, izaje libre.
  - Emergencia manual ⇒ ambos.
- Rearme manual → `RECUPERACION`.



Monitor de comunicaciones (watchdog)

- Persigue el **heartbeat** del supervisor.
- Si el contador supera el límite seguro ⇒ FALLA_WD.
- Dispara la parada de emergencia: la grúa nunca opera con un comando **desactualizado**.



Implementación en CODESYS

De Stateflow a un PLC (IEC 61131)

Para llevar el control a hardware real se traduce la lógica a CODESYS. La consigna pide **dos métodos** y compararlos:

1 — Codificación manual

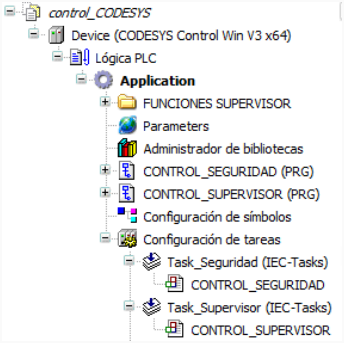
Estructura en **SFC** (Sequential Function Charts) + cálculos en **ST** (Structured Text).

2 — Generación automática

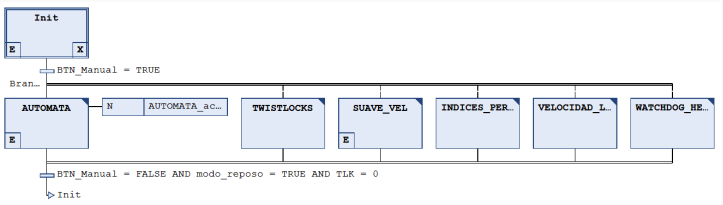
PLC Coder de Simulink: todo a **ST** desde los Statecharts.

Niveles 0 y 1 como POUs cíclicos en paralelo; Nivel 2 en Simulink (simil ST).

Codificación manual: SFC + ST



Árbol del proyecto



Supervisor en SFC (mismos estados y transiciones que Stateflow)

Manual (SFC) vs. autogenerado (ST)

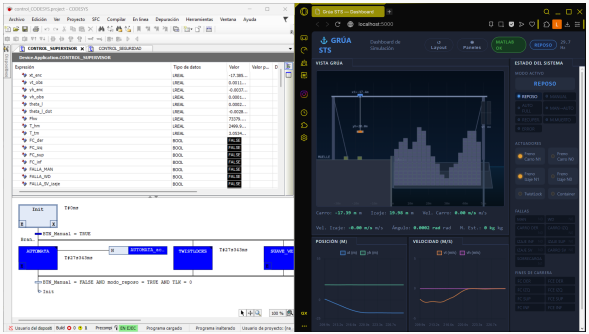
- Rendimiento y carga de CPU **comparables**.
- El ST autogenerado es **monolítico** (un `CASE` global + `IF` anidados) ⇒ poca trazabilidad para diagnóstico en planta.
- Se eligió **SFC manual** para la arquitectura (visual) y ST sólo para los cálculos.

```
22  FUNCTION_BLOCK Nivel
23  VAR_INPUT
24      ssMethodType: SINT;
25      B: LREAL;
26      Joy_XY: ARRAY [0..1] OF LREAL;
27      tita_1: LREAL;
28      theta_1_dot: LREAL;
29      vh_obs: LREAL;
30      vt_obs: LREAL;
31      Fhw: LREAL;
32      yh: LREAL;
33      A: BOOL;
34      xt: LREAL;
35      b_RESET: BOOL;
36      FC_der: BOOL;
37      FC_INF: BOOL;
38      FC_izq: BOOL;
39      FC_sup: BOOL;
40      X: BOOL;
41      FALLA_MAN: LREAL;
42      FALLA_WD: LREAL;
43      FALLA_IZAJE_SV: LREAL;
44      T_hm: LREAL;
```

```
1   CASE ssMethodType OF
2       SS_INITIALIZE:
3           plc_ts_counter1 := 0;
4           (* SystemInitialize for Chart: '<Root>/Nivel 1' *)
5           temporalCounter_i3(timerAction := 1, maxTime := 0);
6           temporalCounter_i1 := 0;
7           temporalCounter_i2 := 0;
8           (* '<S1>:310:7' last_i_ref = -1; *)
9           last_i_ref := -1.0;
10          b_BRK_h_N1 := 0.0;
11          b_BRK_t_N1 := 0.0;
12          dead_zone := 0.0;
13          b_TLK := 0.0;
```

Co-simulación CODESYS ↔ Simulink (OPC-UA)

- PLC virtual (CODESYS) + planta (Simulink) comunicados por **OPC-UA**.
- Software-in-the-Loop: el controlador real maneja la planta simulada.
- Se verifica que responde igual que el modelo original.



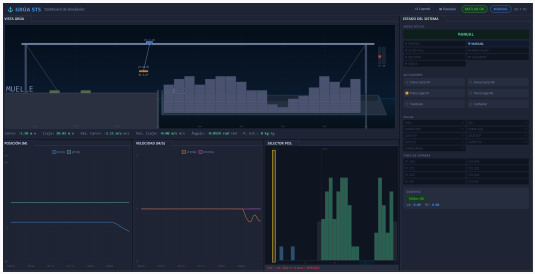
Simulación y resultados

Visualización: animación 2D en tiempo real

- Bloque **UDP** envía el estado de la planta a un programa en **Python**.
- Animación 2D en el navegador: grúa, contenedores, barco y parámetros clave.
- Permite **monitorear y operar** la grúa en los distintos modos.



Modos de operación verificados



Modo manual



Estado de error (parada de emergencia)

Desempeño

- Ciclos completos de carga/descarga **autónomos** y **sin colisiones**.
- Seguimiento de velocidad sin sobrepaso; balanceo residual despreciable al posicionar.
- Transferencia manual↔automático **sin golpe** (*bumpless*).
- Estimación de masa $\Rightarrow$ velocidad a **potencia constante** y bloqueo por sobrecarga.
- Objetivo de productividad: ≥ 30 **contenedores/hora**.

Conclusiones

Conclusiones

Se desarrolló un **sistema híbrido de control jerárquico** que coordina ambos ejes de forma semi-automática y segura.

- **Nivel 2**: lazos PI + observador + compensación de gravedad y anti-sway desacoplado $\Rightarrow$ seguimiento preciso y balanceo amortiguado.
- **Nivel 1**: autómata DEDS que coordina modos, twistlocks, perfil y planificación de trayectorias con evasión de colisiones.
- **Nivel 0**: protección con frenado selectivo y watchdog $\Rightarrow$ seguridad con máxima autoridad.
- **Implementación**: validada en simulación/animación y trasladada a un PLC (CODESYS, SFC/ST) vía OPC-UA.

Trabajos futuros

Hardware real, enlace de menor latencia que OPC-UA, y mejoras de planificación/anti-balanceo para optimizar el tiempo de ciclo.

¿Preguntas?